

学用JDK多线程组件 之



使用CountDownLatch

北京理工大学计算机学院
金旭亮

概述



CountDownLatch是`java.util.concurrent`包中的线程工具类，可以让一个线程阻塞等待多个（其他）线程工作完毕后才开始执行。

CountDownLatch类的主要方法



```
public class CountDownLatch {  
    public CountDownLatch(int count) {}  
    public void await() throws InterruptedException {}  
    public void countDown() {}  
    public long getCount() {}  
}
```



CountDownLatch类的构造函数，需要输入一个初始计数值。工作线程每完成一项任务，就调用一次countDown()方法，让此值减一。



其他线程如调用CountDownLatch.await()方法，会阻塞等待其计数值为0时才能继续执行。



调用CountDownLatch.getCount()方法可用于查询当前的计数值。

示例



```
public class WaitOtherTaskDemo {  
  
    //用于保存工作的结果  
    4 usages  
    static AtomicInteger result = new AtomicInteger( initialValue: 0);  
}
```

在本讲示例中，将创建三个线程完成三项任务，主线程等待这三个线程完成之后，再输出结果。

三项任务分别休眠1、2、3秒，完成之后，分别给全局变量result增加值（1、2和3）

工作任务典型示例

每个工作任务从外界接收一个 `CountDownLatch` 对象，然后干活，活干完之后，调用此对象的 `countDown()` 方法通知外界“活已干完”。

每个工作任务都是在独立的线程中执行的。

右图所示为任务一的代码，其余两个任务的代码高度类似，就是数值不一样而已。

```
1 usage
static void processTask1(CountDownLatch latch) {
    Runnable task = () -> {
        System.out.println("正在执行任务一");
        try {
            TimeUnit.SECONDS.sleep(timeout: 1);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        //将工作结果保存到全局变量中
        result.addAndGet(delta: 1);
        System.out.println("任务一已完成");
        //发出“本任务完成”的通知
        latch.countDown();
    };
    new Thread(task).start();
}
```

主线程代码



```
public static void main(String[] args) throws InterruptedException {  
    //需要等待前面3个任务的完成  
    CountdownLatch prevTasks = new CountdownLatch(3);  
    System.out.println("主线程需要等待三个前置任务的完成");  
    //启动前置任务的执行  
    processTask1(prevTasks);  
    processTask2(prevTasks);  
    processTask3(prevTasks);  
  
    //主线程在此等待  
    prevTasks.await();  
    System.out.println("所有前置工作已完成, 主线程负责收尾");  
    System.out.println("最终的结果为: " + result.get());  
}
```

这三个线程并不等待
CountDownLatch, 而只是
修改它的计数值。

运行截图



```
Run: WaitOtherTaskDemo x
"C:\Program Files\Java\jdk-17\bin\java.exe"
主线程需要等待三个前置任务的完成
正在执行任务一
正在执行任务二
正在执行任务三
任务一已完成
任务二已完成
任务三已完成
所有前置工作已完成，主线程负责收尾
最终的结果为：6
Process finished with exit code 0
```